PYTHON FOUNDATION CHALLENGE

Revision + Progression (Set 4)

Objective: Strengthen foundations, expose gaps, and develop problem-solving ability before OOP by revisiting core concepts with advanced constraints and tackling new algorithmic challenges.

Instructions

- Solve the problems independently. Do not look at previous solutions.
- Focus on producing a working solution first; clean it up afterward if time allows.
- No external libraries unless a question explicitly permits them.
- If you get stuck, identify the Python concept causing the problem rather than immediately searching for a solution.
- Test your code with the examples provided and at least one additional case of your own.

Part A — Previous Questions: Revision (Harder Variants)

Redo these from memory. The purpose is to check retention before moving into the new challenges. Each is a harder variant of a foundational topic you have covered.

1. Prime Number Checker — Optimized

Suggested time: 12–15 minutes

Problem

Given an integer N, determine if it is a prime number. Then, print the sum of all prime numbers from 2 up to N.

Examples

Input: N = 10

```
10 is not prime.
Primes up to 10: [2, 3, 5, 7]
Sum of primes: 17
```

Input: N = 7

```
7 is prime.
Primes up to 7: [2, 3, 5, 7]
Sum of primes: 17
```

Constraints

- You may not use range() to iterate from 2 to N directly for the primality check. You must use a while loop.
- To check if a number is prime, stop checking divisors at its square root (use i * i <= num without importing math).
- Do not use sum().
- Do not import libraries.
- Do not hardcode the answer.

What This Tests: Loops, math logic (square root optimization), nested loops, accumulators, and conditionals.

2. Fibonacci Sequence — Even Sum

Suggested time: 15–20 minutes

Problem

Print the first N numbers of the Fibonacci sequence. Calculate the sum of only the even numbers in that sequence. Finally, print the sequence in reverse order.

Examples

Input: N = 8

```
Sequence: 0, 1, 1, 2, 3, 5, 8, 13
Even Sum: 10 (which is 0 + 2 + 8)
Reversed: 13, 8, 5, 3, 2, 1, 1, 0
```

Constraints

- Generate the sequence using a loop (no recursion).
- Do not store the sequence in an array/list to reverse it. Print the reversed sequence using mathematical logic or string-building without [::-1] or .reverse().
- Do not use sum().
- Do not import libraries.

What This Tests: Variables, state swapping, loops, conditionals, and string/number manipulation.

3. Decimal to Binary Converter

Suggested time: 15–20 minutes

Problem

Given a positive integer, convert it to its binary representation.

Examples

Input: 10 → Output: 1010

Input: 25 → Output: 11001

Input: 1 → Output: 1

Constraints

- Do not use the built-in bin() function.
- Do not use string formatting like f"{num:b}".
- Use the mathematical method: repeatedly divide by 2 and track the remainders.
- Build the final binary string manually using string concatenation.
- Do not import libraries.

What This Tests: Mathematical algorithms, modulo, integer division, string building, and loops.

4. Matrix Multiplication

Suggested time: 20–25 minutes

Problem

Given two matrices, A and B, calculate their product matrix. Matrix multiplication is possible only if the number of columns in A equals the number of rows in B.

Examples

Matrix A: [[1, 2], [3, 4]]

Matrix B: [[5, 6], [7, 8]]

```
Product Matrix:
[19, 22]
[43, 50]
(Explanation: 1×5 + 2×7 = 19; 1×6 + 2×8 = 22, etc.)
```

Constraints

- Use nested loops (three levels of loops are needed).
- Do not import libraries such as numpy.
- Do not use list comprehensions.
- Dynamically build the result matrix based on the dimensions of A and B.
- Do not hardcode the answer.

What This Tests: Advanced nested loops, matrix dimensions, indexing, and accumulators.

5. Remove Spaces & Capitalize Words

Suggested time: 15–20 minutes

Problem

Given a sentence, capitalize the first letter of every word, then create a second version with all spaces removed.

Examples

Input: "hello world from python"

```
Capitalized: "Hello World From Python"
No Spaces: "HelloWorldFromPython"
```

Constraints

- Do not use .replace() to remove spaces.
- Do not use .title() or .capitalize().
- Use loops and conditionals.
- Use ASCII logic to capitalize the first letter (check if the character is between 'a' and 'z', and convert it by subtracting 32 from its ASCII value using ord() and chr()).
- Do not import libraries.

What This Tests: String manipulation, ASCII codes (ord/chr), loops, and flags that track the start of a word.

Part B — New Questions: Progression

These are new problems designed to increase difficulty and reveal gaps in core Python problem-solving skills.

6. Anagram Checker

Suggested time: 15–20 minutes

Problem

Check if two strings are anagrams. An anagram is a word formed by rearranging another word's letters, using all original letters exactly once.

Examples

Input: "listen", "silent" → Anagram

Input: "hello", "world" → Not an anagram

Input: "Dormitory", "dirtyroom" → Anagram

Constraints

- Treat uppercase and lowercase as the same.
- Do not use sort(), sorted(), or set().
- Build a character-frequency dictionary manually for both strings and compare them.
- Do not import libraries.

What This Tests: Dictionaries, frequency counting, string iteration, and comparing data structures.

7. Roman to Integer

Suggested time: 20–25 minutes

Problem

Given a string representing a Roman numeral, convert it to an integer. Symbols: I = 1, V = 5, X = 10, L = 50, C = 100, D = 500, M = 1000. If a smaller numeral appears before a larger one, subtract it (e.g., IV = 4, IX = 9); otherwise, add it.

Examples

Input: "III" → 3

Input: "LVIII" → 58 (L = 50, V = 5, III = 3)

Input: "MCMXCIV" → 1994 (M = 1000, CM = 900, XC = 90, IV = 4)

Constraints

- Store Roman numeral mappings in a dictionary.
- Use a while loop to traverse the string.
- Do not import libraries.
- Do not hardcode the answers.

What This Tests: Dictionaries, string indexing, look-ahead logic, and conditionals.

8. Matrix Rotation (90 Degrees Clockwise)

Suggested time: 20–25 minutes

Problem

Given an N × N 2D matrix, rotate it 90 degrees clockwise in-place (modify the original matrix directly, without creating a brand-new matrix).

Examples

```
matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
]

Output:
[
  [7, 4, 1],
  [8, 5, 2],
  [9, 6, 3]
]
```

Constraints

- In-place only: do not create a new matrix; swap values inside the existing 2D list.
- Hint: first transpose the matrix (swap rows and columns), then reverse each row.
- Do not use [::-1] for reversing.

- Use nested loops.
- Do not import libraries.

What This Tests: 2D list manipulation, in-place swapping, nested loops, and algorithmic problem breakdown.

Progression

The challenge moves from optimized mathematical algorithms → binary conversion → advanced 2D matrix math → ASCII string manipulation → character frequency mapping → dictionary lookups with look-ahead → and finally, in-place 2D grid manipulation.

Time Guide

Section	Questions	Suggested Time
Revision (Harder variants)	1–5	75–90 min total
New progression	6–8	55–70 min total
Overall	**1–8**	**About 2 hours 20 mins**